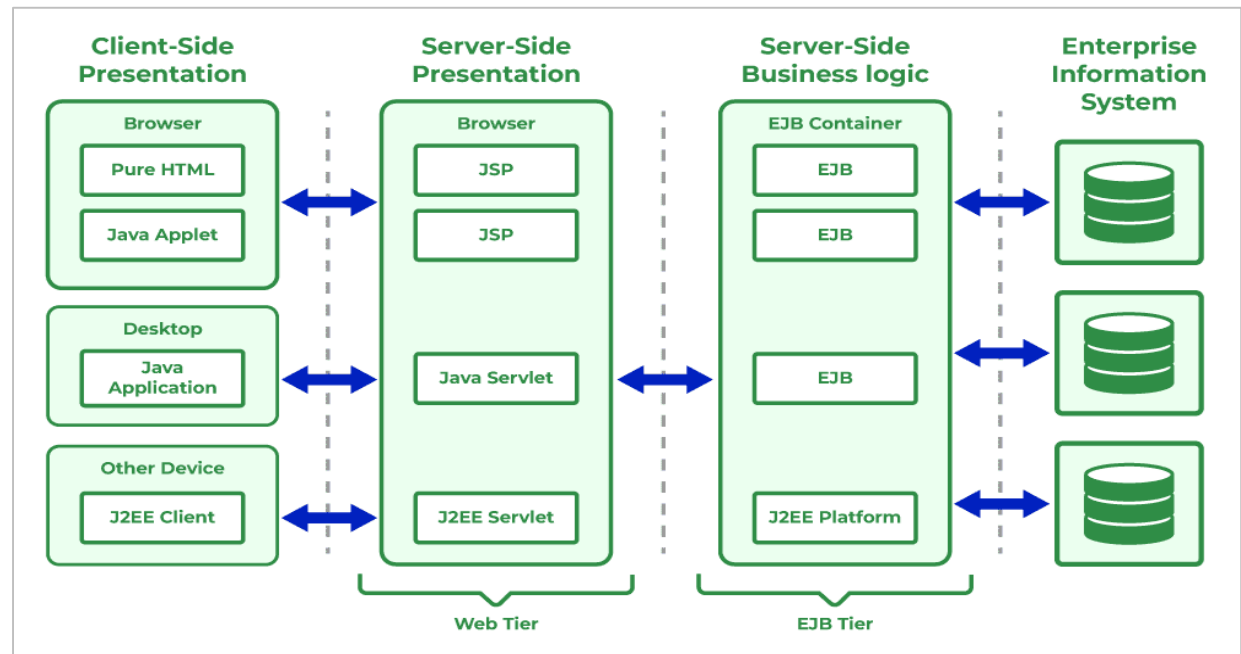


Spring Boot Introduction

Origine de Spring Framework ?

- Pour comprendre l'origine et l'apport du Spring Framework, il faut savoir que son principal auteur, **Rod Johnson**, voulait proposer une alternative au modèle d'architecture logiciel proposé par la plate-forme java **J2EE** (Java 2 Enterprise Edition).
- J2EE est un environnement complexe à appréhender, à cause des environnements serveurs d'application (Servlets, EJB, RMI, ...)
- Les API utilisées par J2EE sont très complexes à exploiter par les développeurs, nécessitant une parfaite connaissance avant toute utilisation.

Architecture J2EE

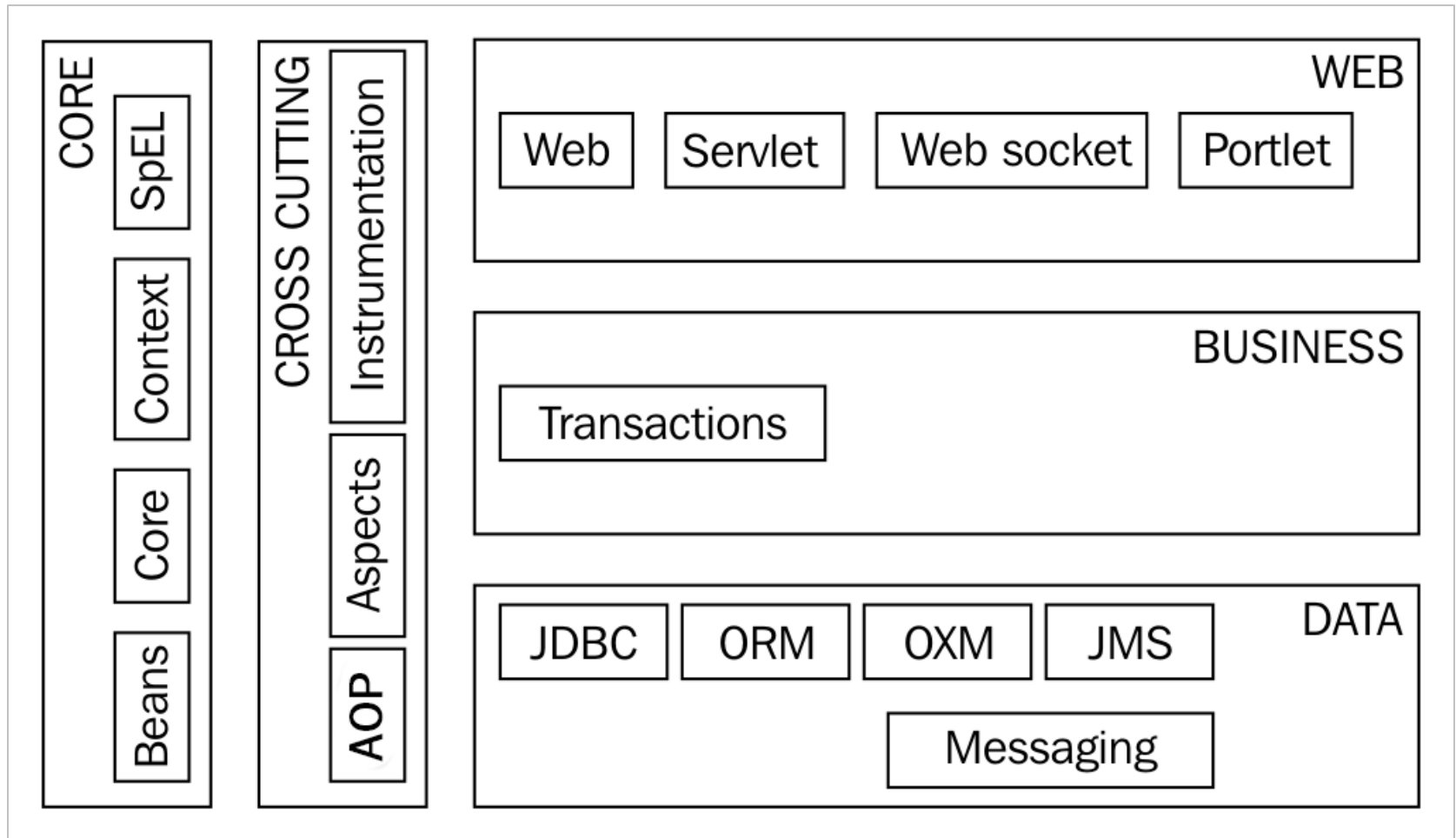


Pourquoi Spring Framework ?

- **Spring Framework** est un framework Backend très largement utilisé dans la communauté Java.
- Il permet d'accélérer le développement d'applications d'entreprise (notamment le développement d'applications Web et d'API Web).
- Spring Framework propose une grande modularité dans le code.
- En fonction des besoins techniques d'une application, il est possible d'incorporer tel ou tel module du Spring Framework et de laisser de côté ceux qui ne sont pas nécessaires.
- Cette approche s'est révélée plus adaptée à des infrastructures cloud et à la mise en place de micro-services.
- Du fait de son niveau d'abstraction et de généralité, SF est parfois difficile de comprendre immédiatement les apports essentiels d'un tel framework. Donc patience.
- Le SF se base sur le principe de l'inversion de contrôle (Inversion of Control ou IoC) et sur la programmation orientée Aspect (Aspect Oriented Programming AOP).
- Il met en œuvre des modèles de conception (Design Patterns) tels que les factories pour fournir un environnement le plus souple possible

Architecture Spring Framework ?

Le Spring Framework est découpé en modules pour faciliter son intégration dans les projets.



Architecture Spring Framework ?

Parmi ces modules, il y a les modules fondamentaux qui font partie du noyau du Spring Framework (*core*) :

Core

Les classes fondamentales utilisées par tous les autres modules

Beans

Le module qui permet de manipuler les objets Java et de créer des *beans*

Context

Ce module introduit la notion de contexte d'application et fournit plusieurs implementations de ces contextes. Avec ce module, il est possible de construire un conteneur léger IoC.

SpEL

Ce module fournit un interpréteur pour le langage d'expression intégré au Spring Framework (*Spring Expression Language* ou *SpEL*).

Les autres modules du Spring Framework permettent majoritairement d'intégrer dans une application des technologies tierces

Qu'est-ce que Spring Boot ?

- **Spring Boot (SB)** fait partie des projets Spring Framework (Spring Data, Spring Cloud, Spring Security, ...)
- SB simplifie le dev d'applications basées sur Spring. Il offre **une configuration automatique et des options par défaut pour réduire la complexité.**
- Les projets SB sont construits au-dessus de SF (donc moins de niveau de complexité)
- SB propose des **configurations par défaut (autoConfiguration)** intelligentes pour accélérer le processus de développement (Contrairement à SF)
- Par exemple, quelques propriétés dans **le fichier yml/.properties** suffisent pour faire connecter ton application (microservice) à une base de données.
- SB utilise les modules de SF et les incorpore comme dépendances dans vos projets, dans un fichier XML (**POM.XML**)
- Il n'est donc pas nécessaire de les connaître pour démarrer le développement d'une application avec SB.
- Par exemple, Spring Web est le module qui permet de créer des applications Web et de les déployer dans un conteneur de servlets Java EE.

Documentation : Spring Boot/Spring Framework

Un autre point fort du Spring Framework (SB) est la qualité de sa documentation.

Commencer par consulter la documentation sur Spring Core :

<https://docs.spring.io/spring-framework/docs/current/reference/html/core.html#spring-core>

et plus généralement la documentation sur les principaux modules du Spring Framework :

<https://docs.spring.io/spring-framework/docs/current/reference/html/index.html>

Pour Spring Boot, vous pouvez vous référer à la documentation officielle :

<https://docs.spring.io/spring-boot/docs/current/reference/html/>

Pour avoir une vision d'ensemble des projets qui existent dans l'éco-système Spring :

<https://spring.io/projects>

Enfin les guides fournissent des réponses pratiques et rapides sur certains points techniques :

<https://spring.io/guides>

Spring Web MVC (Model – View – Controller)

[Spring Web MVC](#) est le module Spring consacré au développement d'application Web et d'API Web.

Le nom de ce module renvoie directement au modèle MVC (Model/View/Controller).

Le modèle MVC

Le modèle [MVC](#) (Modèle Vue Contrôleur) est un modèle d'architecture pour mener à bien la conception d'applications qui nécessitent une interaction de l'utilisateur avec le système.

Il définit trois grandes catégories de responsabilité :

Modèle (Model)

Les classes appartenant à cette catégorie définissent les données applicatives échangées entre l'utilisateur et le système (Base de données) : **Logique métier**

Vue (View)

Les classes appartenant à cette catégorie gèrent la représentation graphique des données et l'interface utilisateur : **Partie visible à l'utilisateur**

Le modèle MVC

Contrôleur (Controller)

Les classes appartenant à cette catégorie gèrent les interactions de l'utilisateur et la mise à jour des vues après la modification des données.

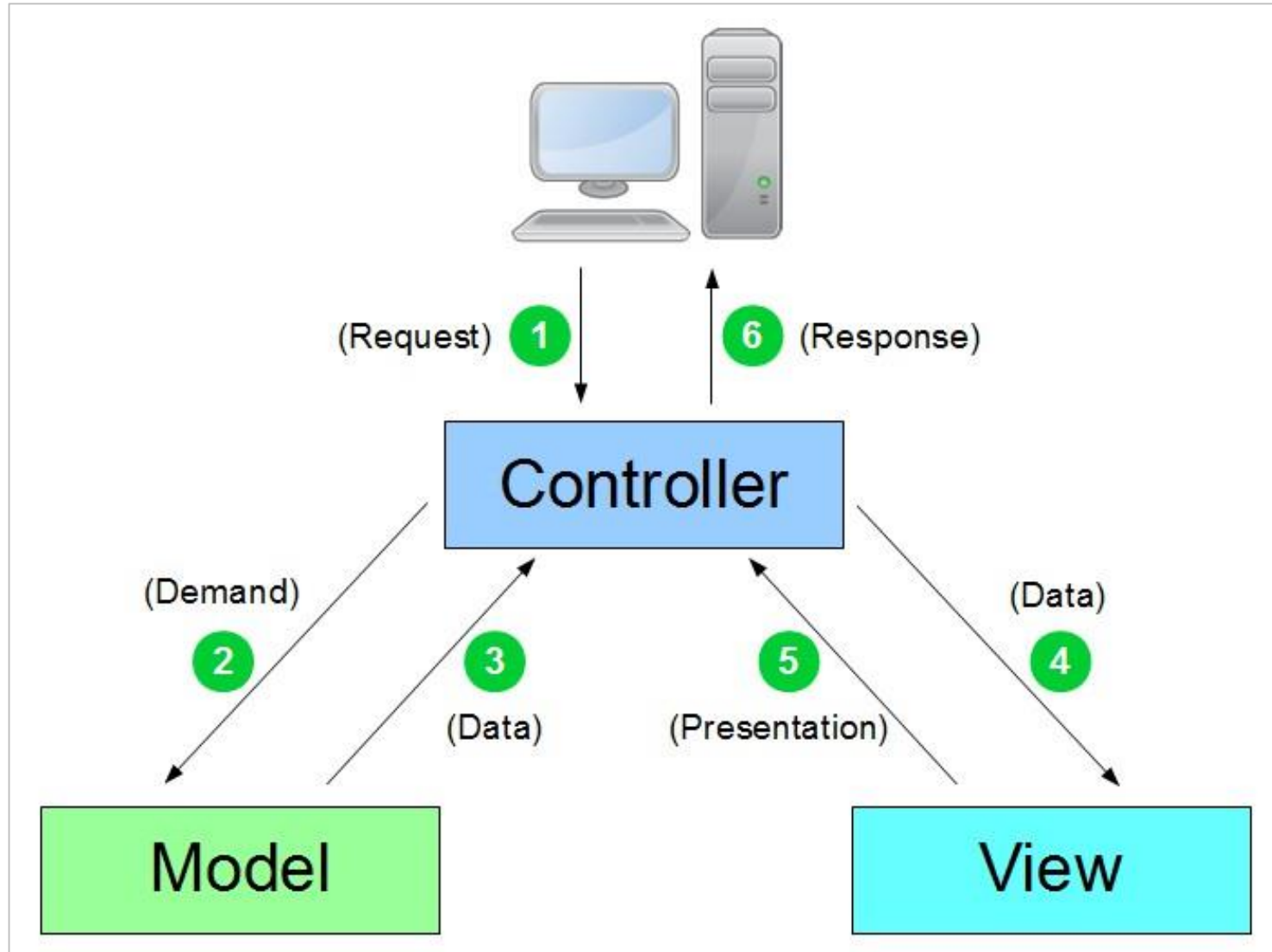
Les contrôleurs assurent la cohérence entre le modèle et la vue.

Dans Spring MVC, les contrôleurs sont généralement implémentés sous forme de classes annotées avec **@Controller**.

Ils définissent **des méthodes mappées aux routes de l'application**, chacune gérant une requête spécifique.

Le contrôleur peut alors interagir avec le modèle, préparer les données nécessaires et les transmettre à la vue correspondante.

Le modèle MVC



Les contrôleurs dans Spring Boot

- Le développement d'une application [Spring Web MVC](#) implique l'implémentation de classes représentant les contrôleurs.
- Dans le modèle MVC, un contrôleur gère les interactions entre l'utilisateur et le système.
- Dans le cadre d'une application Web, les interactions avec le serveur correspondent aux requêtes HTTP émises par le navigateur client.
- Un contrôleur est une classe Java portant l'annotation **@Controller**.
- Pour que le contrôleur soit appelé lors du traitement d'une requête, il suffit d'ajouter l'annotation **@RequestMapping** sur une méthode publique de la classe en précisant la méthode HTTP concernée (par défaut GET) et le chemin d'URI (à partir du contexte de déploiement de l'application) pris en charge par la méthode.

Exemple de contrôleur :

Les contrôleurs dans Spring Boot

```
package iut.mmi;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
public class ItemController {

    @RequestMapping(path="/", method=RequestMethod.GET)
    public String getHome() {
        // ...
        return "index";
    }

    @RequestMapping(path="/item", method=RequestMethod.POST)
    public String addItem() {
        // ...
        return "itemDetail";
    }
}
```

Les contrôleurs dans Spring Boot

- Dans l'exemple ci-dessus, le contrôleur déclare la méthode **getHome** qui traite les requêtes pour le chemin / et la méthode addItem qui traite les requêtes pour le chemin /item.
- La première n'accepte que les requêtes de type GET et la seconde que les requêtes de type POST.
- Il existe les annotations suivantes :

@GetMapping, **@PutMapping**, **@PostMapping**, **@DeleteMapping**, **@PatchMapping** qui fonctionnent comme l'annotation @RequestMapping sauf qu'il n'est pas nécessaire de préciser la méthode HTTP concernée puisqu'elle est mentionnée dans le nom de l'annotation :

Les contrôleurs dans Spring Boot

```
package iut.mmi;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class ItemController {

    @GetMapping(path="/")
    public String getHome() {
        // ...
        return "index";
    }

    @PostMapping(path="/item")
    public String addItem() {
        // ...
        return "itemDetail";
    }

}
```

Mise en place d'un projet SB

Pour générer un projet SB, on utilise le Spring initializr <https://start.spring.io/>, sans ajouter de dépendance





Project

- ☐ Gradle - Groovy
- ☐ Gradle - Kotlin
- ☒ Maven

Language

- ☒ Java
- ☐ Kotlin
- ☐ Groovy

Spring Boot

- ☐ 4.0.0 (SNAPSHOT)
- ☐ 4.0.0 (M3)
- ☐ 3.5.7 (SNAPSHOT)
- ☒ 3.5.6
- ☐ 3.4.11 (SNAPSHOT)
- ☐ 3.4.10

Project Metadata

Group

Dependencies

ADD DEPENDENCIES... CTRL + B

No dependency selected



GENERATE CTRL + G

EXPLORE CTRL + SPACE

...

Mise en place d'un projet SB

Ajouter les dépendances de votre application : -

 **spring**initializr

Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Java

☐ Kotlin

☐ Groovy

☒ Maven

Language

☐ 4.0.0 (SNAPSHOT)

☐ 4.0.0 (M3)

☐ 3.5.7 (SNAPSHOT)

☒ 3.5.6

Spring Boot

☐ 3.4.11 (SNAPSHOT)

☐ 3.4.10

Project Metadata

Group

iut.mmi

Artifact

webAppli

Name

webAppli

Description

Application Web

Package name

iut.mmi.webAppli

Packaging

☒ Jar

☐ War

Java

☒ 25

☐ 21

☐ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Boot DevTools

DEVELOPER TOOLS

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA

SQL

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

MariaDB Driver

SQL

MariaDB JDBC and R2DBC driver.

GENERATE

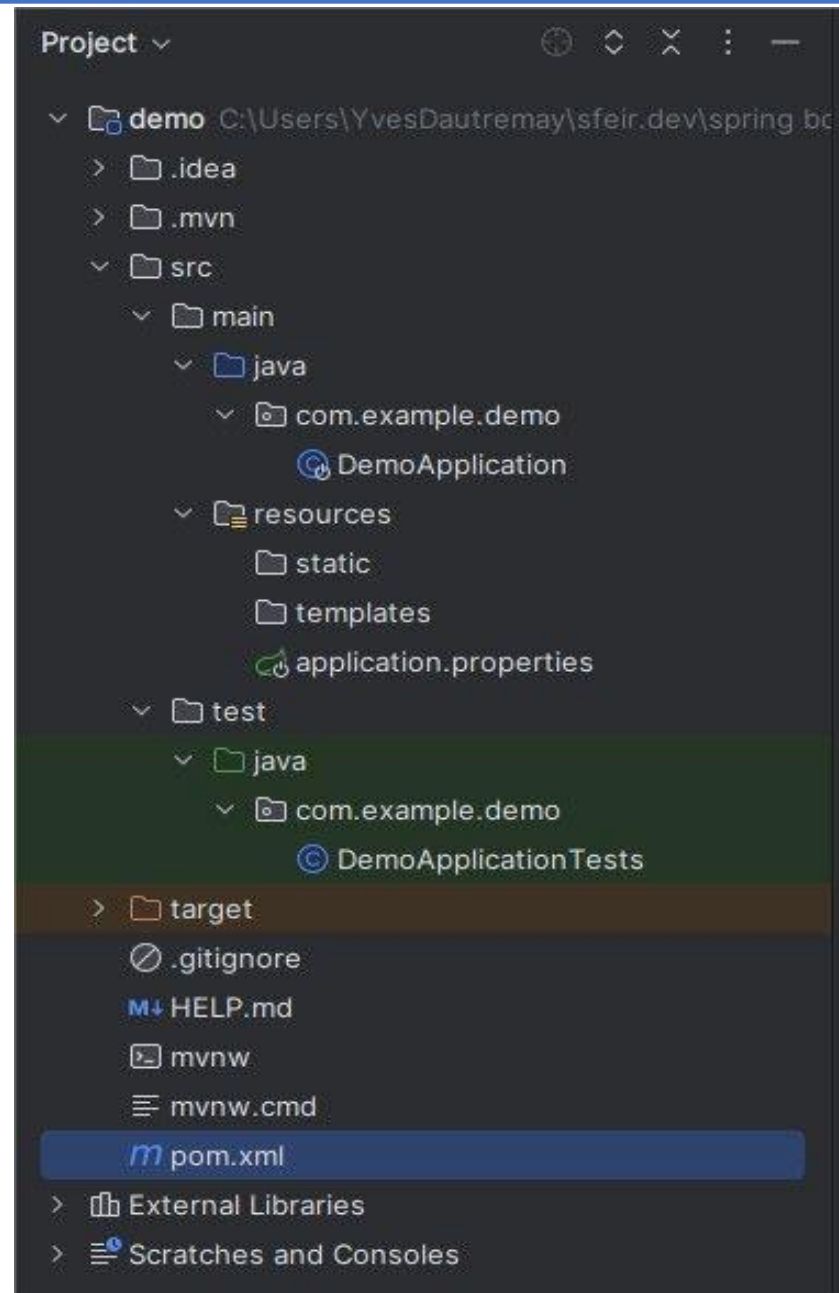
CTRL + G

EXPLORE

CTRL + SPACE

...

Structure d'un projet



Mise en place d'un projet SB

Fichier des dépendances POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.6</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>iut.mmi</groupId>
  <artifactId>webAppli</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>webAppli</name>
  <description>Application Web</description>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
```